



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS 226 - Concurrent Systems

Practical 4 Specification

Lock Granularity and Concurrent Lists

Release Date: 14 September 2026

Submission Deadline: 27 September 2026

1 Overview

In this practical, you will investigate how the granularity of locking affects the behaviour of a concurrent data structure.

A simple way to make a data structure thread-safe is to protect the entire structure using a single lock. This is known as *coarse-grained synchronisation*. Although this approach is simple, only one thread can access the protected data structure at a time.

An alternative is *fine-grained synchronisation*, where smaller parts of the data structure are protected by separate locks. In a linked list, individual nodes may be protected by their own locks. This allows multiple threads to potentially operate on different parts of the list at the same time.

The practical consists of three tasks. You will first implement a coarse-grained concurrent linked list using a single lock. You will then implement a fine-grained version using node-level locking and *hand-over-hand locking*. Finally, you will experimentally compare the performance of the two implementations.

The emphasis of this practical is on understanding the trade-off between locking overhead and the amount of concurrency permitted by a locking strategy.

2 Learning Outcomes

By completing this practical you should be able to:

- Explain the difference between coarse-grained and fine-grained synchronisation.
- Implement a concurrent linked list using a single coarse-grained lock.
- Implement node-level locking in a concurrent linked list.
- Explain and implement hand-over-hand locking.
- Explain why neighbouring nodes may need to be locked simultaneously.

3 Requirements

This practical is to be completed in groups of **three**. Group members are expected to contribute equally to the implementation, and all group members must be able to explain any part of the solution during the demonstration.

Starter code containing the linked-list node structure and a **Main** class will be provided.

The list represents a sorted set of integer values and provides the following operations:

- `add(int value)` – adds a value if it is not already present.
- `remove(int value)` – removes a value if it is present.
- `contains(int value)` – determines whether a value is present.

The provided node structure must not be replaced with a different data structure.

Java's `ReentrantLock` must be used for the locking mechanisms required in this practical.

4 Task 1 – Coarse-Grained Synchronisation

Implement the `CoarseList` class.

The entire linked list must be protected using a **single lock**.

Your implementation must provide the following methods:

- `add(int value)`
- `remove(int value)`
- `contains(int value)`

Each operation must acquire the list lock before accessing the linked list and release the lock when the operation is complete.

Your implementation must:

- Preserve the sorted order of the linked list.
- Prevent duplicate values from being inserted.
- Ensure mutual exclusion when accessing the list.
- Correctly release the lock regardless of whether the operation succeeds or fails.

You are encouraged to make use of a `try-finally` block to ensure that locks are always released.

5 Task 2 – Fine-Grained Synchronisation

Implement the `FineList` class.

Unlike the implementation from Task 1, the fine-grained implementation must not use a single lock to protect the entire linked list.

Instead, each node contains its own lock.

While traversing the list, your implementation must use **hand-over-hand locking**.

Your `FineList` must implement:

- `add(int value)`
- `remove(int value)`

- `contains(int value)`

Your implementation must:

- Preserve the sorted order of the linked list.
- Prevent duplicate values from being inserted.
- Use node-level locks rather than a single global list lock.
- Use hand-over-hand locking when traversing the list.
- Correctly release all acquired locks.

When adding or removing a node, ensure that both the current node and its predecessor are appropriately protected before modifying the structure of the list.

6 Task 3 – Lock Granularity Experiment

You will experimentally compare the `CoarseList` and `FineList` implementations. The provided `Main` class performs a mixture of:

- `add()` operations,
- `contains()` operations, and
- `remove()` operations.

Run the same workload using both implementations. At a minimum, test the following thread configurations:

- 2 threads
- 4 threads
- 8 threads
- 16 threads

Each implementation must perform the same number and type of operations. Present your results in a table similar to the following:

Threads	Coarse-Grained Time	Fine-Grained Time
2		
4		
8		
16		

7 Demonstration

Students are required to demonstrate their implementation to a tutor during their scheduled practical session.

During the demonstration, students must:

- Demonstrate the `CoarseList` implementation.
- Demonstrate the `FineList` implementation.
- Identify where the single coarse-grained lock is acquired and released.
- Identify where hand-over-hand locking occurs in the fine-grained implementation.
- Demonstrate the comparison experiment.
- Show the recorded execution times for both implementations.
- Explain the observed experimental results.

Students should be prepared to explain their implementation rather than simply demonstrate that the program executes successfully.

In particular, students may be asked to identify specific sections of their code and explain why locks are acquired or released at those locations.

Conceptual questions will be asked during the demonstration.

8 Upload Checklist

Before submitting, ensure that your group has included all of the following files:

- `Node.java`
- `CoarseList.java`
- `FineList.java`
- `Main.java`
- The results of your comparison experiment.

9 Submission

Your solution will be submitted via **ClickUp**. **All group members are required to submit.**

Your submission should include:

- The source code mentioned in the upload checklist.
- The student numbers of all group members.
- The results of the lock granularity experiment.

All required source files must be placed in a single ZIP archive. The archive must be named according to the following format: `uXXXXXXXX.zip` where `XXXXXXXX` is your student number.

No late submissions will be accepted!

10 Marking Rubric

Assessment Item	Marks
Coarse-Grained List implementation	2
Fine-Grained List implementation	2
Lock granularity experiment and results	2
Conceptual Question 1	3
Conceptual Question 2	3
Conceptual Question 3	3
Total	15